

Universidad Tecnológica de Querétaro

Ingeniería en Desarrollo y Gestión de Software

Materia: Desarrollo para Dispositivos Inteligentes

**Evaluación 2: Ecosistema Completo (Teléfono +
Wearable + Smart TV)**

**Documento: Reporte de Configuración de
Herramientas y Emuladores**

Alumno: Mauricio Gabriel Vazquez

Matrícula: 2023371007

Proyecto: TravelApp - Kiosco Inteligente de Abordaje

Agosto 2026

Índice

1. Configuración de Herramientas (SA.6.A)	2
1.1. Versiones de SDK	2
1.2. Entornos de Desarrollo (IDEs) y Plugins	2
1.3. Dependencias Clave (pubspec.yaml y package.json)	2
1.4. Pasos de Instalación Reproducibles	3
2. Configuración de Emuladores (SA.6.B)	3
2.1. Emulador de Teléfono Inteligente	3
2.2. Emulador Wear OS	3
2.3. Simulador Smart TV (Chrome DevTools)	3
2.4. Evidencia Visual de Emuladores	7
3. Registro de Problemas (Troubleshooting)	7

Índice de figuras

1. Emulador de wearable corriendo	4
2. Configuración de wearable	4
3. SmartTV corriendo	5
4. SmartTV corriendo	5
5. Teléfono corriendo	6

1. Configuración de Herramientas (SA.6.A)

Para asegurar la reproducibilidad del proyecto, se detallan a continuación las herramientas, lenguajes y frameworks utilizados en el ecosistema TravelApp.

1.1. Versiones de SDK

Al ejecutar el comando `flutter --version` en la terminal, se obtiene el siguiente entorno de desarrollo:

- **Flutter SDK:** Versión 3.44.0
- **Dart SDK:** Versión 3.12.0
- **OS:** Windows 11

1.2. Entornos de Desarrollo (IDEs) y Plugins

Se utilizaron dos entornos de desarrollo paralelos para el manejo del frontend (móvil/wearable) y el backend/PWA:

- **Android Studio:** Versión Koala — 2024.1.1
- **Plugins de Android Studio:** Flutter (v79.0.2), Dart (v233.11899).
- **Visual Studio Code:** Versión 1.90.0
- **Extensiones de VS Code:** Live Server, Prisma, ESLint.
- **Herramientas Multimedia:** ffmpeg versión 7.0.1 (utilizado para optimización de assets de la Unidad 3).

1.3. Dependencias Clave (pubspec.yaml y package.json)

El ecosistema depende de las siguientes librerías críticas para su funcionamiento:

- **Frontend (Flutter):**
 - `http: ^1.2.0` (Peticiones API REST)
 - `provider: ^6.1.1` (Gestión de estado)
 - `socket_io_client: ^3.0.0` (Telemetría bidireccional en tiempo real)
- **Backend (Node.js):**
 - `express: ^4.18.2`
 - `socket.io: ^4.7.4`
 - `prisma: ^5.10.2`

1.4. Pasos de Instalación Reproducibles

Para que otro desarrollador pueda replicar este entorno desde cero, debe seguir estos pasos exactos:

1. Clonar el repositorio raíz: `git clone https://github.com/mauriciogabrielvazquez/Evaluaci`
2. **Servidor:** Ingresar a la carpeta del backend, ejecutar `npm install`, configurar el archivo `.env` con las credenciales Prisma/JWT, e iniciar con `node server.js`.
3. **PWA (Smart TV):** Actualizar la IP local en `app.js` y desplegar el `index.html` mediante Live Server en el puerto 5500.
4. **Apps (Teléfono y Wearable):** Ingresar a los directorios respectivos, ejecutar `flutter pub get` para instalar dependencias, configurar la IP del servidor en el `baseUrl` y lanzar las aplicaciones a los emuladores con `flutter run`.

2. Configuración de Emuladores (SA.6.B)

2.1. Emulador de Teléfono Inteligente

- **Modelo:** Moto G85
- **Nivel de API (OS):** API 34 (Android 14)
- **RAM Asignada:** 4096 MB

2.2. Emulador Wear OS

- **Forma:** Round (Redondo)
- **Nivel de API:** Wear OS API 33 (Android 13)
- **RAM Asignada:** 1024 MB

2.3. Simulador Smart TV (Chrome DevTools)

Para emular el comportamiento de una pantalla 10-foot, se utilizó Google Chrome DevTools con la siguiente configuración estricta:

- **Resolución Mínima:** 1920×1080 píxeles.
- **Device Pixel Ratio (DPR):** 1.0
- **User Agent Modificado:** Generic Smart TV / WebOS

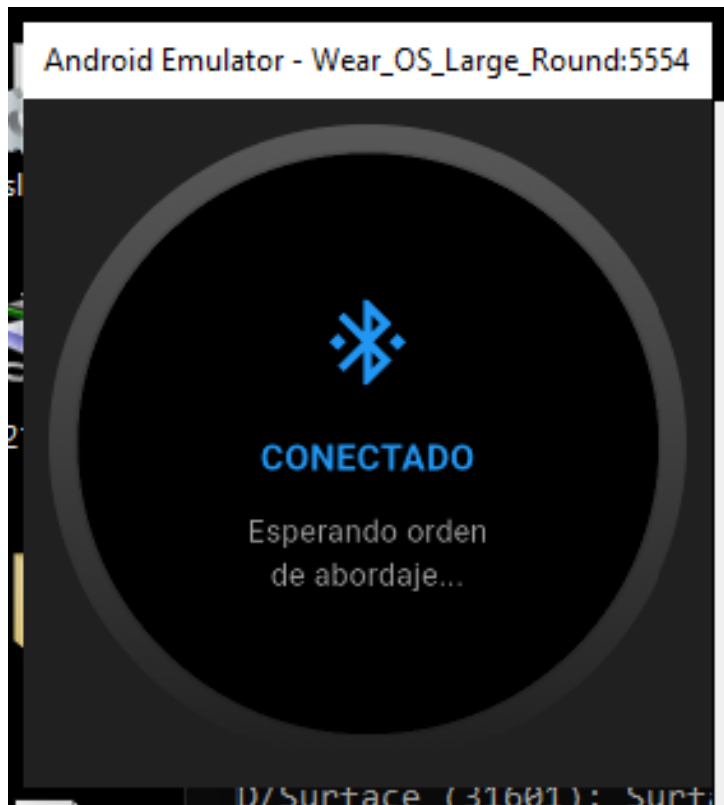


Figura 1: Emulador de werable corriendo

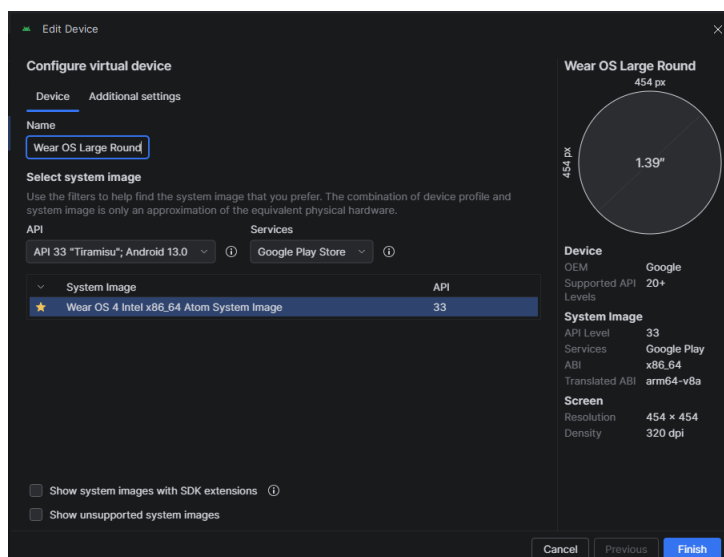


Figura 2: Configuracion de werable

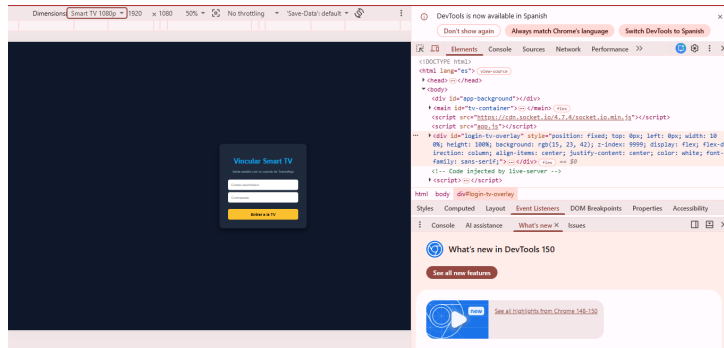


Figura 3: SmartTV corriendo



Figura 4: SmartTV corriendo

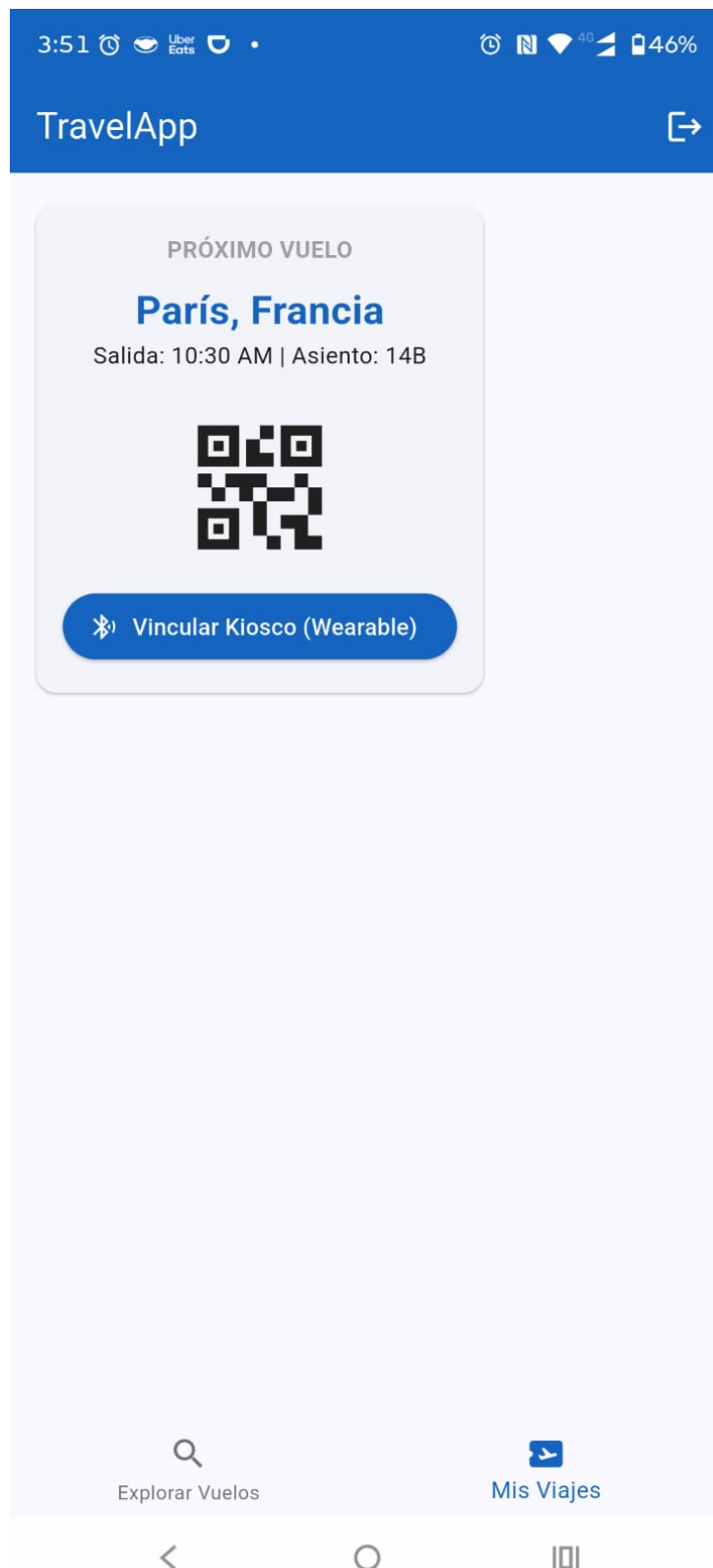


Figura 5: Telefono corriendo

2.4. Evidencia Visual de Emuladores

3. Registro de Problemas (Troubleshooting)

Durante la integración del ecosistema, se presentó y resolvió la siguiente incidencia técnica documentada:

Problema: Al intentar compilar la aplicación para el Wearable, la consola de Flutter lanzó un error fatal indicando `java.util.concurrent.TimeoutException` seguido de `Gradle task assembleDebug failed with exit code 1`, deteniendo por completo el emulador. Al intentar detener el proceso por consola (`.gradlew --stop`), se generó un error secundario de `JAVA_HOME is not set`.

Solución Aplicada: El error se diagnosticó como un proceso *zombie* de Java que bloqueaba los archivos del proyecto debido a la falta de variables de entorno globales. Para solucionarlo:

1. Se forzó el cierre del proceso `OpenJDK Platform binary` directamente desde el Administrador de Tareas de Windows, liberando la memoria bloqueada.
2. Se purgó la caché corrupta de compilación mediante el comando `flutter clean`.
3. Se reconstruyó el árbol de dependencias con `flutter pub get` y se lanzó nuevamente el emulador, logrando una compilación exitosa.